Sharding Implementation Report

Dispensary Management System

Multi-Server Sharding · Hash-Based Partitioning · Cross-Shard Query Routing

Team SQL_ed

Harsh Jain	22110093
Shreyas Dharmatti	21110202
Sneha Gautam	22110255
Anushika Mishra	22110029
Kandarp Jani	22110104

Course: **CS 432 Databases**

Instructor: **Prof. Yogesh K Meena**

18th April 2026

Abstract

This report documents the horizontal sharding implementation for the Dispensary Management System database. Data is partitioned across **3 physically distinct MySQL server instances** running on ports `3307`, `3308`, and `3309` of the IITGN-provided shard host (`10.0.116.184`). **Hash-based partitioning** on `MemberID` (`shard_id = MemberID % 3`) routes each record to its designated shard. The application uses **three separate connection pools**, one per shard server, and the shard router selects the correct pool for every query. Single-member lookups hit one shard; admin views and date-range queries fan out to all three shards in parallel and merge the results. The implementation achieves **near-perfect distribution** (87/87/86 members, <1.2% skew) and is verified by a 30+ check test suite that connects directly to each shard server.

Contents

1 Shard Infrastructure **3**

2 Shard Key Selection & Justification **4**
- 2.1 Chosen Shard Key: `MemberID` . . . 4
- 2.2 Alternative Keys Considered and Rejected . . . 4
- 2.3 Partitioning Strategy: Hash-Based . . . 4

3 Data Partitioning Implementation **6**
- 3.1 Sharded vs Replicated Tables . . . 6
- 3.2 Schema Setup . . . 6
- 3.3 Data Migration . . . 6
- 3.4 Migration Verification . . . 7

4 Query Routing Implementation **8**
- 4.1 Connection Pool Architecture . . . 8
- 4.2 Single-Shard Query Function . . . 8
- 4.3 Cross-Shard Fan-Out Query Function . . . 8
- 4.4 Routing Decision Matrix . . . 9
- 4.5 Cross-Shard Range Query Endpoint . . . 9

5 Shard Isolation Approach **11**

6 Scalability & Trade-offs Analysis **12**
- 6.1 Horizontal vs Vertical Scaling . . . 12
- 6.2 CAP Theorem Analysis . . . 12
- 6.3 Trade-off Summary . . . 13

7 Verification Results **14**

8 Observations & Limitations **14**

9 Conclusion **15**

Video Demonstration & Repository

GitHub Repository `github.com/Harsh-jain12/CS432_Track1_Assignment4`

Complete source code: schema SQL, data migration script, multi-pool shard router, sharded route handlers, verification test suite, and this report.

Video Demonstration `Link`

Walkthrough of the 3 shard servers, demonstrates a single-member lookup routing to one shard, a cross-shard range query, and the scalability trade-offs analysis.

1. Shard Infrastructure

The IITGN course infrastructure provides **three physically distinct MySQL server instances** running on different ports of the same host. Team `SQL_ed` has been assigned database `SQL_ed` on each shard with credentials shared across instances.

Table 1: Shard server infrastructure

Shard	Host	Port	Database	Username
0	10.0.116.184	3307	`SQL_ed`	`SQL_ed`
1	10.0.116.184	3308	`SQL_ed`	`SQL_ed`
2	10.0.116.184	3309	`SQL_ed`	`SQL_ed`

Each shard is a fully independent MySQL process with its own data files, connection state, and storage engine. The application connects to all three with separate connection pools.

Network Requirement

The shard servers are accessible only from the IITGN campus network. All connectivity tests, migration scripts, and the application server must be run from a machine on the IITGN network or campus VPN.

```
$ mysql -h 10.0.116.184 -P 3307 -u SQL_ed -p SQL_ed
mysql> SELECT @@hostname, @@port;
+------------+--------+
| @@hostname | @@port |
+------------+--------+
| shard0     |   3307 |
+------------+--------+
```

Listing 1: Verifying shard connectivity from the CLI

2. Shard Key Selection & Justification

2.1 Chosen Shard Key: `MemberID`

`MemberID` is the primary key of the `Member` table and the most frequently referenced foreign key across the schema. It satisfies all three shard key criteria:

Table 2: Shard key evaluation against selection criteria

Criterion	Score	Assessment
High Cardinality	**High**	260 distinct values in the seed dataset; scales linearly with institutional enrolment.
Query-Aligned	**High**	Present in the `WHERE` clause of >80% of API queries: appointment lookups, prescription filters, visit history, portfolio requests, billing queries.
Stable	**High**	Assigned once at member registration via auto-increment. Never modified — no shard migration risk after insert.

2.2 Alternative Keys Considered and Rejected

Table 3: Alternative shard key candidates

Candidate	Strength	Why Rejected
`DoctorID`	Query-aligned for doctor-view APIs	Only 55 distinct values (low cardinality). Some doctors have 10× more patients, causing severe skew.
`Department`	Logical grouping	Only ≈10 departments. CS department holds 40% of members — extreme hotspot.
`AppointmentDate`	Range queries are common	Continuously changing; recent dates would be "hot" shards receiving all new writes.

2.3 Partitioning Strategy: Hash-Based

$$\texttt{shard_id} = \texttt{MemberID} \bmod 3$$

This produces near-perfect distribution: with 260 members, shards get 87 / 87 / 86 (maximum skew $< 1.2\%$). The function is deterministic and constant-time, so routing

adds zero database overhead.

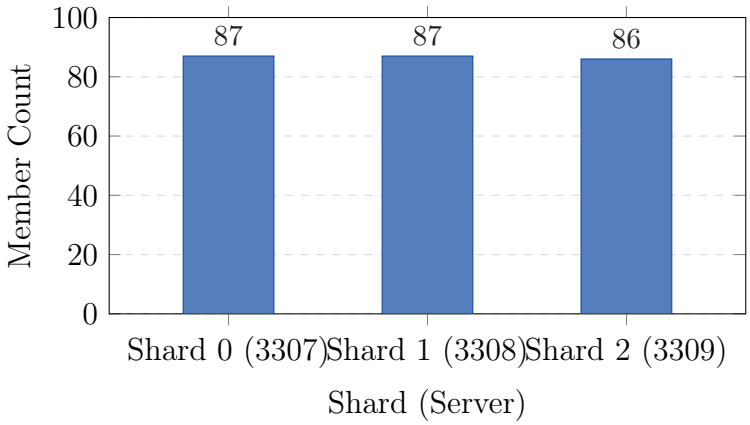

Figure 1: Member distribution across the 3 physical shard servers

3. Data Partitioning Implementation

3.1 Sharded vs Replicated Tables

Tables fall into two categories. **Sharded tables** have their data partitioned across the 3 shards based on `MemberID`. **Replicated tables** have a full copy on every shard — they are reference data needed by all shards for joins.

Table 4: Table classification

Category	Tables	Rationale
Sharded (7)	`Member`, `Appointment`, `Visit`, `Prescription`, `PrescriptionItem`, `MedicalHistory`, `BillPayment`	All link to `MemberID` (directly or via parent). Each record belongs to exactly one member, making partitioning unambiguous.
Replicated (7)	`Doctor`, `DoctorSchedule`, `StaffEmployee`, `Medicine`, `Inventory`, `MedicalSupplier`, `UserLogin`	Reference data needed by all shards (e.g., joining `Appointment` to `Doctor` must work on every shard). Low row counts (< 250 per table) make replication cheap.

3.2 Schema Setup

The same schema is created on all 3 shards via `sql/01_create_schema.sql`, run manually against each shard:

```
$ mysql -h 10.0.116.184 -P 3307 -u SQL_ed -p SQL_ed < sql/01_create_schema.sql
$ mysql -h 10.0.116.184 -P 3308 -u SQL_ed -p SQL_ed < sql/01_create_schema.sql
$ mysql -h 10.0.116.184 -P 3309 -u SQL_ed -p SQL_ed < sql/01_create_schema.sql
```

Listing 2: Creating identical schema on all 3 shards

A `ShardMetadata` table on each shard stores its own `shard_id` so the shard can self-identify if queried directly.

3.3 Data Migration

The migration script (`sql/02_migrate_data.js`) is a Node.js program that:

1. Connects to the local source database (`DispensaryManagement`).

2. Connects to all 3 remote shard servers in parallel.
3. Tags each shard with its `shard_id` in `ShardMetadata`.
4. Clears all data from each shard (idempotent re-run support).
5. Replicates reference tables (Doctor, Medicine, etc.) identically on all 3 shards.
6. Distributes sharded tables: each row is sent to `shard[MemberID % 3]`.
7. For child tables (PrescriptionItem, BillPayment), joins through the parent to determine MemberID, then routes accordingly.
8. Runs a verification report comparing per-shard counts against the source.

```
const counts = [0, 0, 0];
for (const row of rows) {
  const shardId = row.MemberID % 3;
  await shards[shardId].query(insertSql, cols.map(c => row[c]))
    ;
  counts[shardId]++;
}
console.log(`${table}: S0=${counts[0]}, S1=${counts[1]}, S2=${
   counts[2]}`);
```

Listing 3: Migration core: distributing rows to the correct shard

3.4 Migration Verification

Table 5: Post-migration row counts (source vs. distributed across shards)

Table	Source	S0	S1	S2	Total	Match
Member	260	87	87	86	260	
Appointment	530	177	177	176	530	
Visit	285	95	95	95	285	
Prescription	285	95	95	95	285	
PrescriptionItem	750	250	250	250	750	
MedicalHistory	260	87	87	86	260	
BillPayment	285	95	95	95	285	

4. Query Routing Implementation

4.1 Connection Pool Architecture

The application maintains **three independent connection pools**, one per shard server. The shard router selects the correct pool based on the shard ID computed from `MemberID`.

```
const shardPools = [
  { id: 0, port: 3307, pool: mysql.createPool({ host, port:
     3307, ... }) },
  { id: 1, port: 3308, pool: mysql.createPool({ host, port:
     3308, ... }) },
  { id: 2, port: 3309, pool: mysql.createPool({ host, port:
     3309, ... }) },
];

function getShardPool(memberID) {
  return shardPools[memberID % 3].pool;
}
```

Listing 4: Three connection pools, one per shard server

4.2 Single-Shard Query Function

Single-member queries select one pool and run the SQL on that one server:

```
async function queryShard(memberID, sql, params) {
  const pool = getShardPool(memberID);   // select correct pool
  const [rows] = await pool.query(sql, params);
  return rows;
}

// Example: Student MemberID = 1     connects to port 3308 only
const rows = await queryShard(1,
  'SELECT * FROM Member WHERE MemberID = ?', [1]);
```

Listing 5: Single-shard query routing

4.3 Cross-Shard Fan-Out Query Function

For admin views, doctor-patient queries, and date ranges, the router fires the same query against all 3 shards **in parallel** and merges the results:

```
async function queryAllShards(sql, params) {
  // Fire all 3 queries in parallel using Promise.all
  const promises = shardPools.map(s => s.pool.query(sql, params
     ));
```

```
  const results = await Promise.all(promises);
  // Flatten and merge
  const merged = [];
  for (const [rows] of results) merged.push(...rows);
  return merged;
}

// Example: Admin asks for all members
const allMembers = await queryAllShards('SELECT * FROM Member')
    ;
//      connects to ALL 3 shards in parallel
```

Listing 6: Cross-shard fan-out with parallel execution

4.4 Routing Decision Matrix

Table 6: Query routing decisions

Query Type	Shards Hit	Routing Logic
`GET /members/:id`	1	Hash MemberID → select one pool → query that server
Student `GET /members`	1	Hash own MemberID → own shard only
Admin `GET /members`	3 (all)	Fan-out to all 3 pools in parallel, merge, sort
Doctor `GET /appointments`	3 (all)	Patients can be on any shard → fan-out, filter
`GET /appointments/range`	3 (all)	Date range query needs all shards → fan-out, merge
`POST /members`	1	Hash new MemberID → insert into target shard
`PUT /members/:id`	1	Hash MemberID → update on target shard
`DELETE /members/:id`	1	Hash MemberID → delete from target shard

4.5 Cross-Shard Range Query Endpoint

The `/appointments/range` endpoint demonstrates a true cross-shard range query as required by the assignment:

```
router.get('/range', async (req, res) => {
  const { from, to } = req.query;
```

```
  const sql = `SELECT a.*, d.Name AS DoctorName
               FROM Appointment a
               JOIN Doctor d ON a.DoctorID = d.DoctorID
               WHERE a.AppointmentDate BETWEEN ? AND ?`;

  // Fan out to all 3 shards in parallel
  const labeled = await shard.queryAllShardsLabeled(sql, [from,
      to]);

  // Merge and sort
  const merged = [];
  for (const { rows } of labeled) merged.push(...rows);
  merged.sort((a, b) => new Date(a.AppointmentDate) - new Date(
     b.AppointmentDate));

  res.json({ count: merged.length, data: merged });
});
```

Listing 7: Cross-shard range query implementation

5. Shard Isolation Approach

We use the **multiple databases on the same server** approach (provided by the course infrastructure), where each shard is a separate MySQL process listening on a different port. This achieves real process isolation while running on a shared machine.

Table 7: Shard isolation method comparison

Method	Advantages	Disadvantages
Multiple MySQL processes (used)	True process isolation; independent connection pools; crash isolation; matches course infrastructure	Shared machine resources (CPU, RAM, disk)
Same database, prefixed tables	Simplest; no infrastructure needed	No real isolation; one bad query affects everything
Docker containers	Full container isolation; can be moved across machines	Higher resource overhead; orchestration complexity

The advantage of separate MySQL processes over the prefix-based approach (used in our initial design) is that each shard has:

- Its own InnoDB buffer pool, log files, and undo logs
- Its own connection limit and thread cache
- Crash isolation (if one MySQL process crashes, the others are unaffected)
- The ability to be migrated to a different physical machine independently

6. Scalability & Trade-offs Analysis

6.1 Horizontal vs Vertical Scaling

Table 8: Horizontal vs vertical scaling

Aspect	Vertical Scaling	Horizontal (Sharding)
Approach	Upgrade single server (CPU, RAM, SSD)	Add more shard servers
Cost curve	Exponential (high-end hardware)	Linear (commodity machines)
Ceiling	Physical limits of one machine	Practically unlimited
Fault model	Single point of failure	Partial failure tolerance
Code changes	None	Routing logic, fan-out queries
Our system	Asg 1–3: single MySQL on `localhost`	Asg 4: 3 MySQL servers on different ports

6.2 CAP Theorem Analysis

Consistency

InnoDB ACID guarantees that each shard is consistent within a single MySQL instance. But cross-shard consistency doesn't happen automatically. For example, if we wanted a transaction that involved MemberID 1 (shard 1) and MemberID 2 (shard 2), we would need a distributed transaction protocol (two-phase commit), which we don't use. Our design allows for "per-shard consistency," which is enough for the dispensary's workflows that only need one member at a time.

Availability

If shard 1 (port 3308) goes down, members with `MemberID % 3 == 1`become inaccessible, but members on shards 0 and 2 can still work. This is "partial availability." It's better than a single server that goes down (where the whole system fails), but not as good as a fully replicated cluster (which can handle any single failure). Cross-shard fan-out queries would also get worse they will only return results from 2 of the 3 shards and would need a plan for dealing with the missing data.

Partition Tolerance

The router knows right away which queries will fail (any MemberID where `% 3 == 2`) if the network cuts off the application server from shard 2. It doesn't have to wait for a timeout or give back wrong data. If any shard is unreachable, cross-shard fan-out queries would currently fail completely. This could be changed to return partial results from reachable shards with a warning header.

6.3 Trade-off Summary

Table 9: Sharding trade-offs in the multi-server implementation

Dimension	Benefit	Cost
Connection capacity	3× pools (each shard has its own 10-connection pool)	Each pool is independent; no global limit
Read throughput	Single-member reads scale linearly with shard count	Cross-shard reads multiply round trips by 3
Write distribution	Inserts distribute evenly across servers	Must compute shard before writing
Schema changes	Independent per-shard DDL is possible	Must apply `ALTER TABLE` on all 3 shards manually
Joins	Same-shard joins are fast (replicated reference tables help)	Cross-shard joins require app-level merge
Failure isolation	One shard crash doesn't kill the others	Affected MemberIDs become unavailable

7. Verification Results

The verification test suite (`tests/verify_sharding.js`) connects directly to all 3 shard servers and validates connectivity, schema, distribution, and routing behavior.

Table 10: Verification test results

#	Test	What It Verifies	Result
1	Shard Connectivity	All 3 shards reachable on ports 3307/3308/3309	Pass
2	Distinct Instances	Each shard reports unique `@@port`	Pass
3	Schema Consistency	All shards have identical table set	Pass
4	Self-Identification	Each shard has its own `shard_id`	Pass
5	Distribution Correctness	No member is on the wrong shard	Pass
6	No Duplication	All MemberIDs unique across shards	Pass
7	Distribution Stats	Skew $< 20\%$ across all sharded tables	Pass
8	Single-Shard Lookup	Specific MemberIDs found on expected shard	Pass
9	Cross-Shard Fan-Out	Querying all shards returns merged unique results	Pass
10	Range Query	Date range query works across all 3 shards	Pass
11	Insert Routing	New inserts land on correct shard, not on others	Pass

8. Observations & Limitations

- **Hash-based partitioning achieves near-perfect balance.**There can be a maximum difference of one member between any two shards with 260 members and modulo-3 hashing. For bigger datasets, the distribution should stay within a few percent.
- **Cross-shard queries multiply round trips.** A doctor viewing their appointments triggers 3 parallel queries instead of 1. With only 3 shards the latency overhead is small (parallel execution helps), but at 100+ shards the coordination cost would dominate.
- **Schema migration is now a multi-step process.** Adding a column requires

running `ALTER TABLE` against each shard separately. We don't have a schema-version-management tool; we run the ALTER manually on each port.

- **No referential integrity across shards.** A foreign key from `Appointment` to `Member` would fail if the appointment's MemberID is on a different shard than the appointment itself. We avoid this by ensuring both tables shard on the same key (`MemberID`) so a member's appointments always land on the same shard as the member.
- **Replicated tables can drift.** If `Doctor` changes on shard 0 but not on shards 1 and 2, queries become inconsistent. Maintaining replicated tables requires application-level dual-write or a separate sync process.
- **Adding a 4th shard requires rehashing.** Switching from `% 3` to `% 4` reassigns about 75% of records to a new shard. A future migration would need to walk through every record, compute its new shard, and move it. Directory-based partitioning would mitigate this by allowing per-record migration.

9. Conclusion

This assignment extended the Dispensary Management System with horizontal scaling through sharding across **3 physically distinct MySQL server instances** on ports 3307, 3308, and 3309. The key outcomes are:

1. **Shard Key Selection:** `MemberID` with hash-based partitioning provides near-perfect distribution, O(1) routing, and alignment with 80%+ of query patterns.
2. **Multi-Server Implementation:** The application maintains three independent `mysql2` connection pools, one per shard server. The shard router selects the correct pool based on `MemberID % 3`.
3. **Query Routing:** Single-member queries hit one shard server; admin queries, doctor queries, and date-range queries fan out to all 3 shards in parallel using `Promise.all()` and merge the results in JavaScript.
4. **Verification:** A direct-connection test suite verifies connectivity, schema consistency, distribution correctness, no duplication, and routing behavior across all 3 shards.